15 · Routing Basics

Intermediate Angular learning notes

a. Definition

The **Angular Router** maps URLs to components, so a single-page app can still have many 'pages'. You define an array of **routes** (`path` + `component`), register it with `provideRouter(routes)`, place a `<router-outlet />` where the page should appear, and use `routerLink` for navigation without a reload.

A route with `path: '**'` matches everything else and is perfect for a *404 Not Found* page. `routerLinkActive` adds a CSS class to the link of the page you are currently on.

b. Diagram



URL -> matching route -> component rendered in <router-outlet>

c. Syntax

```
const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent, title: 'About' },
  { path: '**', component: NotFoundComponent },
];

provideRouter(routes)           // in app.config.ts

<a routerLink="/about" routerLinkActive="active">About</a>
<router-outlet />
```

d. Example Program

Programs/15_Routing_Basics/router-basics.component.ts

```
import { ApplicationConfig, Component } from '@angular/core';
import { provideRouter, RouterLink, RouterLinkActive, RouterOutlet, Routes, withHashLocation }
from '@angular/router';

// Three simple "pages"
@Component({ selector: 'app-home', template: `<h2>Home</h2><p>Welcome to Koyambedu Bus Stand.
</p>` })
export class HomeComponent {}

@Component({ selector: 'app-about', template: `<h2>About</h2><p>We run buses across Tamil
Nadu.</p>` })
export class AboutComponent {}

@Component({ selector: 'app-not-found', template: `<h2>404 - Page not found</h2>` })
export class NotFoundComponent {}
```

```ts
// Route table: path -> component. '**' (wildcard) must be LAST.
const routes: Routes = [
  { path: '', component: HomeComponent, title: 'Home' },
  { path: 'about', component: AboutComponent, title: 'About' },
  { path: '**', component: NotFoundComponent },
];

// In a real app this lives in app.config.ts (withHashLocation is only to keep URLs simple
here)
export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes, withHashLocation())],
};

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <nav>
      <a routerLink="/" routerLinkActive="active" [routerLinkActiveOptions]="{ exact: true
}">Home</a> |
      <a routerLink="/about" routerLinkActive="active">About</a> |
      <a routerLink="/nowhere">Broken link</a>
    </nav>
    <router-outlet />
  `,
  styles: `.active { font-weight: bold; }`,
})
export class AppComponent {}
```

e. Output

Rendered in the browser (text on screen, after the UI steps below):

1. The app opens on the Home page.
2. Click **About**.
3. Click **Broken link**.

```
--- Home page ---
Home | About | Broken link
Home
Welcome to Koyambedu Bus Stand.

--- After clicking About ---
Home | About | Broken link
About
We run buses across Tamil Nadu.

--- After clicking a broken link ---
Home | About | Broken link
404 - Page not found
```

f. Real-World Example (English + தமிழ்)

English

Koyambedu bus stand is one station, but each platform number leads to a different bus. A URL is the platform number; the matching component is the bus.

தமிழ்

கோயம்பேடு பேருந்து நிலையத்தை நினைத்துப் பாருங்கள். நிலையம் ஒன்றுதான்; ஆனால் 'மதுரை - நடைமேடை 5', 'கோவை - நடைமேடை 8' என்று ஒவ்வொரு நடைமேடைக்கும் ஒரு எண் உண்டு. அந்த எண்ணுக்குப் போனால் அந்த ஊருக்கான பேருந்து நிற்கும்.

Router அந்த அறிவிப்புப் பலகை போல: URL (நடைமேடை எண்) → component (பேருந்து). இல்லாத எண்ணுக்குப் போனால் 'அப்படி ஒரு நடைமேடை இல்லை' என்று காட்டும் - அதுதான் `**` route (404).

g. Pros

- Deep links and browser back/forward work
- Lazy loading support
- Guards and resolvers built in

h. Cons

- Misconfigured routes are hard to debug
- Hash vs path URLs need server setup

i. Advanced / New Features

- Standalone `provideRouter`
- View transitions (`withViewTransitions`)
- Functional guards
- `redirectTo` can be a function (v18.1)

j. Common Mistakes

- Forgetting `<router-outlet />`; routes match but nothing appears.
- Using `href="/about"` instead of `routerLink`, which reloads the entire page.
- Placing the `**` wildcard route before other routes (it matches first).
- Forgetting to import `RouterLink` / `RouterOutlet` into a standalone component.

k. Interview Questions

Q1. What is the Angular Router?
A library mapping URLs to components, enabling navigation in a single-page app without reloads.

Q2. `routerLink` vs `href`?
`routerLink` navigates inside the SPA (no reload); `href` makes the browser request the page again.

Q3. What does `path: '**'` do?
It is the wildcard route that matches any URL not matched earlier, used for a 404 page.

Q4. What is `routerLinkActive`?
A directive that adds a CSS class to a link when its route is active.

I. Practice Exercises

1. Create Home, Products and Contact pages with a navigation bar.
2. Add a redirect: `{ path: 'home', redirectTo: '', pathMatch: 'full' }`.
3. Highlight the active link using `routerLinkActive` and a CSS class.